

COMPUTER NETWORKS - ASSIGNMENT 2

BY – VEDANT MUNDADA

BT2024268

GITHUB-

[HTTPS://GITHUB.COM/HAXOUS742/RDT-TAHOE-RENO-SIMULATION](https://github.com/HAXOUS742/RDT-TAHOE-RENO-SIMULATION)

Libraries Needed – matplotlib, random, math, colorama, time, argparse

RDT 2.2 SIMULATION-

1. Background-

Reliable Data Transfer (RDT) protocols form the foundation of dependable communication over unreliable networks. The RDT protocol family presents an incremental approach to solving communication challenges, with each version addressing additional problems. RDT 2.2 specifically handles the issue of both data corruption and out-of-order packet delivery through sequence numbering, checksums, and acknowledgment mechanisms.

2. Project Overview-

This project implements a simulation of the RDT 2.2 protocol in Python. The implementation consists of four main components:

- Client: Responsible for sending data packets and processing acknowledgments.
- Server: Responsible for receiving data packets, validating them, and sending acknowledgments
- Unreliable Channel: Simulates a network channel with configurable corruption probability
- Main Program: Coordinates the simulation by transferring user-inputted text between client and server

The simulation demonstrates how RDT 2.2 ensures reliable data transfer despite an unreliable communication channel.

3. System Architecture:

- Main.py - Entry point that coordinates the simulation
- Client.py - Implements the sender functionality
- Receiver.py - Implements the receiver functionality
- Unreliable_channel.py - Simulates the network channel with corruption

4. How to run:

- In the cmd, you have to write the following command to run all the files at once:
 - “python main.py --run”

5. How to operate:

- First it will give you a input area to write a sentence (make sure that the number of words are more that 4, for it to work properly), which it will then parse into separate words and then send each word as a separate packet.

```
PS C:\Users\VedantM\Desktop\GIT\RD-Tahoe-Reno-Simulation> cd .\RDT_2.2\  
PS C:\Users\VedantM\Desktop\GIT\RD-Tahoe-Reno-Simulation\RDT_2.2> python .\main.py --run  
Enter a sentence to send: I like Computer Networks very much
```

- Then it will start the simulation and show how the server and client will interact with each other:

```
Data to send: ['I', 'like', 'Computer', 'Networks', 'very', 'much']  
  
[Client]: Initialized with seq_num 0  
[Server]: Initialized with expected_seq_num 0  
  
Starting RDT 2.2 simulation...
```

```

----- SENDING NEW DATA: 'I' -----
Channel corrupted DATA packet: seq_num flipped
[Client]: Sent packet with seq_num 0, payload: 'I', checksum: 147
[Server]: Received packet with seq_num 1, payload: 'I', checksum: 147, corrupt: True
[Server]: Packet is corrupted
Channel corrupted ACK packet
[Server]: Sent ACK with seq_num 1, checksum: 0
[Client]: Received corrupted ACK
[Client]: Retransmitting packet with seq_num 0
[Server]: Received packet with seq_num 0, payload: 'I', checksum: 147, corrupt: False
[Server]: Packet seq_num 0 is correct and in order
[Server]: Delivered to upper layer: 'I'
[Server]: Sent ACK with seq_num 0, checksum: 255
[Server]: Expected_seq_num updated to 1
[Client]: Received correct ACK for seq_num 0
[Client]: seq_num updated to 1

```

```

----- SENDING NEW DATA: 'like' -----
[Client]: Sent packet with seq_num 1, payload: 'like', checksum: 240
[Server]: Received packet with seq_num 1, payload: 'like', checksum: 240, corrupt: False
[Server]: Packet seq_num 1 is correct and in order
[Server]: Delivered to upper layer: 'like'
[Server]: Sent ACK with seq_num 1, checksum: 0
[Server]: Expected_seq_num updated to 0
[Client]: Received correct ACK for seq_num 1
[Client]: seq_num updated to 0

```

```

----- SENDING NEW DATA: 'Computer' -----
[Client]: Sent packet with seq_num 0, payload: 'Computer', checksum: 153
[Server]: Received packet with seq_num 0, payload: 'Computer', checksum: 153, corrupt: False
[Server]: Packet seq_num 0 is correct and in order
[Server]: Delivered to upper layer: 'Computer'
[Server]: Sent ACK with seq_num 0, checksum: 255
[Server]: Expected_seq_num updated to 1
[Client]: Received correct ACK for seq_num 0
[Client]: seq_num updated to 1

```

```

----- SENDING NEW DATA: 'Networks' -----
[Client]: Sent packet with seq_num 1, payload: 'Networks', checksum: 168
[Server]: Received packet with seq_num 1, payload: 'Networks', checksum: 168, corrupt: False
[Server]: Packet seq_num 1 is correct and in order
[Server]: Delivered to upper layer: 'Networks'
Channel corrupted ACK packet
[Server]: Sent ACK with seq_num 1, checksum: 0
[Server]: Expected_seq_num updated to 0
[Client]: Received corrupted ACK
[Client]: Retransmitting packet with seq_num 1
Channel corrupted DATA packet: seq_num flipped
[Server]: Received packet with seq_num 0, payload: 'Networks', checksum: 168, corrupt: True
[Server]: Packet is corrupted
[Server]: Sent ACK with seq_num 1, checksum: 0
[Client]: Received correct ACK for seq_num 1
[Client]: seq_num updated to 0

```

```

----- SENDING NEW DATA: 'very' -----
[Client]: Sent packet with seq_num 0, payload: 'very', checksum: 16
[Server]: Received packet with seq_num 0, payload: 'very', checksum: 16, corrupt: False
[Server]: Packet seq_num 0 is correct and in order
[Server]: Delivered to upper layer: 'very'
Channel corrupted ACK packet
[Server]: Sent ACK with seq_num 0, checksum: 255
[Server]: Expected_seq_num updated to 1
[Client]: Received corrupted ACK
[Client]: Retransmitting packet with seq_num 0
[Server]: Received packet with seq_num 0, payload: 'very', checksum: 16, corrupt: False
[Server]: Packet is out-of-order
[Server]: Sent ACK with seq_num 0, checksum: 255
[Client]: Received correct ACK for seq_num 0
[Client]: seq_num updated to 1

```

```

----- SENDING NEW DATA: 'much' -----
[Client]: Sent packet with seq_num 1, payload: 'much', checksum: 248
[Server]: Received packet with seq_num 1, payload: 'much', checksum: 248, corrupt: False
[Server]: Packet seq_num 1 is correct and in order
[Server]: Delivered to upper layer: 'much'
[Server]: Sent ACK with seq_num 1, checksum: 0
[Server]: Expected_seq_num updated to 0
[Client]: Received correct ACK for seq_num 1
[Client]: seq_num updated to 0

SIMULATION COMPLETE!!

```

6. Working and Colour code:

- So basically all the packets will be send one after the other, and as it is a stop and wait protocol, it will move ahead only when the previous packet has reached.
- The initial sending and receiving are represented in **BLUE**.
- The normal sending and receiving are represented in **GREEN**.
- The receiving of corrupt packets is represented in **RED**.
- The sending of corrupt packets is represented in **YELLOW**.

7. Code explanation and structure:

- Packet structure:

```

class Packet:
    def __init__(self, type, seq_num, payload="", checksum=None):
        self.type = type
        self.seq_num = seq_num
        self.payload = payload
        if checksum is None:
            self.checksum = self.compute_checksum()
        else:
            self.checksum = checksum

```

- Function to calculate checksum:

```

def compute_checksum(self):
    data = f"{self.type}{self.seq_num}{self.payload}".encode()
    checksum = 0
    for byte in data:
        checksum += byte
    return checksum % 256

def is_corrupt(self):
    return self.checksum != self.compute_checksum()

```

- Function to send and corrupt ACKs and Packets:

```

def send_to_server(self, packet):
    corrupted_packet = Packet(packet.type, packet.seq_num, packet.payload, packet.checksum)
    rand = random.random()
    if rand < self.corruption_prob:
        if corrupted_packet.type == "DATA":
            if random.choice([True, False]):
                corrupted_packet.seq_num = 1 - corrupted_packet.seq_num
                print(f"{Fore.YELLOW}          Channel corrupted DATA packet: seq_num flipped{Style.RESET_ALL}")
            else:
                if corrupted_packet.payload:
                    idx = random.randint(0, len(corrupted_packet.payload) - 1)
                    corrupted_packet.payload = (corrupted_packet.payload[idx] +
                                                chr(ord(corrupted_packet.payload[idx]) + 1) +
                                                corrupted_packet.payload[idx+1:])
                    print(f"{Fore.YELLOW}          Channel corrupted DATA packet: payload altered{Style.RESET_ALL}")
                elif corrupted_packet.type == "ACK":
                    corrupted_packet.seq_num = 1 - corrupted_packet.seq_num
                    print(f"{Fore.YELLOW}          Channel corrupted ACK packet{Style.RESET_ALL}")
    self.to_server_queue.put(corrupted_packet)

```

```

def send_to_client(self, packet):
    corrupted_packet = Packet(packet.type, packet.seq_num, packet.payload, packet.checksum)
    if random.random() < self.corruption_prob:
        if corrupted_packet.type == "ACK":
            old_seq = corrupted_packet.seq_num
            corrupted_packet.seq_num = 1 - corrupted_packet.seq_num
            print(f"{Fore.YELLOW}          Channel corrupted ACK packet {Style.RESET_ALL}")
    self.to_client_queue.put(corrupted_packet)

```

- Function to receive packets:

```

def receive_from_client(self):
    if not self.to_server_queue.empty():
        return self.to_server_queue.get()
    return None

```

```

def receive_from_server(self):
    if not self.to_client_queue.empty():
        return self.to_client_queue.get()
    return None

```

- The real function for receiving and sending the packets and acks are written in client.py and server.py and main.py, is just to combine all of this and run it all together as a whole.
- Also the probability can be adjusted from here:

```
class UnreliableChannel:  
    def __init__(self, corruption_prob=0.9):  
        self.to_server_queue = queue.Queue()  
        self.to_client_queue = queue.Queue()  
        self.corruption_prob = corruption_prob
```

RDT 3.0 SIMULATION-

1. Background:

This report presents a Python-based implementation of the Reliable Data Transfer 3.0 (RDT 3.0) protocol. RDT 3.0 extends the functionality of RDT 2.2 by adding mechanisms to handle packet loss in addition to data corruption. The implementation simulates a client-server communication model over an unreliable channel that can both corrupt and lose packets. Key protocol features include sequence numbering, checksums, timeouts, acknowledgments, and retransmission mechanisms. The report details the implementation, protocol functionality, and performance analysis in lossy network conditions.

2. Project Overview-

This project implements a simulation of the RDT 3.0 protocol in Python. The implementation consists of four main components:

- Client: Responsible for sending data packets, processing acknowledgments, and implementing timeouts for lost packets
- Server: Responsible for receiving and validating data packets and sending acknowledgments
- Unreliable Channel: Simulates a network with configurable corruption and packet loss probabilities
- Main Program: Coordinates the simulation by transferring user-inputted text between client and server

The key advancement over RDT 2.2 is the implementation of timeout-based retransmission mechanisms to recover from lost packets, making the protocol robust against both corruption and packet loss.

3. How to run:

- In the cmd, you have to write the following command to run all the files at once:
 - “python main.py --run”

4. How to operate:

- First it will give you a input area to write a sentence (make sure that the number of words are more that 4, for it to work properly), which it will then parse into separate words and then send each word as a separate packet.

```
SIMULATION COMPLETE!!  
PS C:\Users\VedantM\Desktop\GIT\RDT-Tahoe-Reno-Simulation\RDT_3.0> python main.py --run  
Enter a sentence to send: I like computer networks very much
```

- Then it will start the simulation and show how the server and client will interact with each other:

Ex:

```
Data to send: ['I', 'like', 'computer', 'networks', 'very', 'much']  
  
[Client]: Initialized with seq_num 0  
[Server]: Initialized with expected_seq_num 0  
  
Starting RDT 3.0 simulation...
```

```
----- SENDING NEW DATA: 'like' -----  
Channel lost DATA packet  
[Client]: Sent packet with seq_num 1, payload: 'like', checksum: 240  
[Server]: No packet received yet  
[Client]: No ACK received yet  
[Server]: No packet received yet  
[Client]: No ACK received yet  
[Server]: No packet received yet  
[Client]: No ACK received yet  
[Client]: Timeout occurred, retransmitting packet with seq_num 1  
Channel corrupted DATA packet: seq_num flipped  
[Server]: Received packet with seq_num 0, payload: 'like', checksum: 240, corrupt: True  
[Server]: Packet is corrupted  
[Server]: Sent ACK with seq_num 0, checksum: 255  
[Client]: Received out-of-order ACK  
[Client]: Retransmitting packet with seq_num 1  
[Server]: Received packet with seq_num 1, payload: 'like', checksum: 240, corrupt: False  
[Server]: Packet seq_num 1 is correct and in order  
[Server]: Delivered to upper layer: 'like'  
Channel: Lost ACK packet  
[Server]: Sent ACK with seq_num 1, checksum: 0  
[Server]: Expected_seq_num updated to 0  
[Client]: No ACK received yet  
[Client]: Timeout occurred, retransmitting packet with seq_num 1  
Channel lost DATA packet  
[Server]: No packet received yet  
[Client]: No ACK received yet  
[Server]: No packet received yet  
[Client]: No ACK received yet  
[Client]: Timeout occurred, retransmitting packet with seq_num 1  
Channel corrupted DATA packet: payload altered  
[Server]: Received packet with seq_num 1, payload: 'lile', checksum: 240, corrupt: True  
[Server]: Packet is corrupted  
[Server]: Sent ACK with seq_num 1, checksum: 0  
[Client]: Received correct ACK for seq_num 1  
[Client]: seq_num updated to 0
```

5. Working and Colour code:

- So basically all the packets will be send one after the other, and as it is a stop and wait protocol, it will move ahead only when the previous packet has reached.
- The initial sending and receiving are represented in **BLUE**.
- The normal sending and receiving are represented in **GREEN**.
- The receiving of corrupt packets and packet loss is represented in **RED**.
- The sending of corrupt packets and packet loss is represented in **YELLOW**.

6. Code explanation and structure:

- The structure is exactly the same, except the fact the the unreliable channel now will also loose packets from time to time, and to handle them we have introduced a timer:

```
def send_to_server(self, packet):
    if random.random() < self.loss_prob:
        print(f"{Fore.RED}          Channel lost DATA packet{Style.RESET_ALL}")
    else:
        corrupted_packet = Packet(packet.type, packet.seq_num, packet.payload, pack
```

```
def send_to_client(self, packet):
    if random.random() < self.loss_prob:
        print(f"{Fore.RED}          Channel: Lost ACK packet{Style.RESET_ALL}")
    else:
```

- The timer amount can be adjusted from here:

```
init(autoreset=True)

class Client:
    def __init__(self, channel):
        self.channel = channel
        self.seq_num = 0
        self.current_packet = None
        self.waiting_for_ack = False
        self.send_time = 0
        self.timeout = 0.15 # seconds
        print(f"{Fore.BLUE}[Client]: Initialized with seq_num {self.seq_num}{Style.RESET_ALL}")
```

- The probabilities can be adjusted from here:

```
class UnreliableChannel:
    def __init__(self, corruption_prob=0.1, loss_prob=0.1):
        self.to_server_queue = queue.Queue()
        self.to_client_queue = queue.Queue()
        self.corruption_prob = corruption_prob
```

TCP_RENO_SIMULATION-

1. Background:

This report provides a detailed analysis of a Python-based simulation for TCP Reno congestion control. The simulation demonstrates the behavior of TCP Reno's congestion window (cwnd) under various network conditions, including packet loss events through timeout and triple duplicate acknowledgments. The report includes a thorough explanation of the implementation, visualization of results, and analysis of the algorithm's performance across different scenarios.

2. TCP Reno Algorithm Overview:

TCP Reno implements four key mechanisms:

- Slow Start: Exponential growth of the congestion window until reaching the slow start threshold (ssthresh)
- Congestion Avoidance: Linear growth of the congestion window after reaching ssthresh
- Fast Retransmit: Detecting packet loss through triple duplicate ACKs
- Fast Recovery: Adjusting congestion window after detecting loss

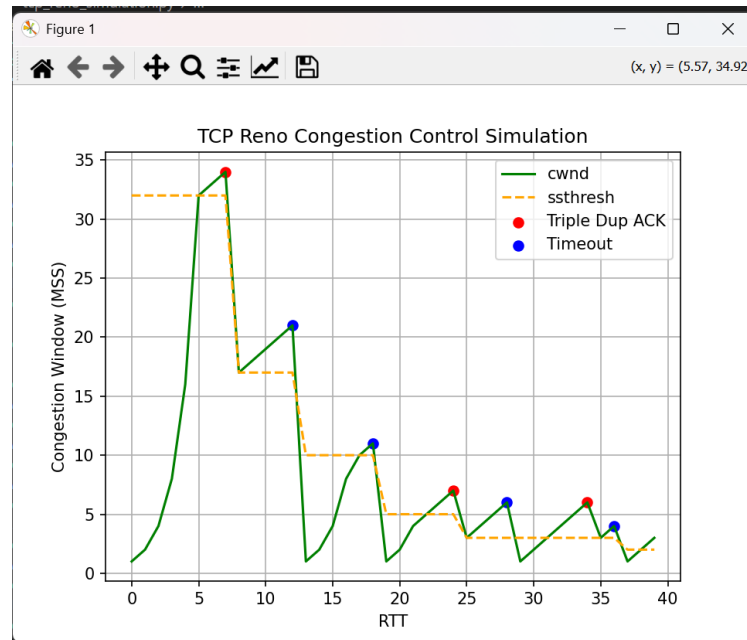
The algorithm transitions between states based on network feedback, particularly acknowledgments (ACKs) and timeouts

3. How to operate:

- When you run the code you will have to input the following parameters:

```
PS C:\Users\VedantM\Desktop\GIT\RD-Tahoe-Reno-Simulation> python -u "c:\Users\VedantM\TCP_Reno\tcp_reno_simulation.py"
Enter RTT (in seconds):
3
Enter MSS size (in bytes):
2
Enter initial ssthresh (in MSS):
32
Enter probability of loss events per RTT during congestion avoidance (0 to 1):
0.3
Enter number of RTTs to simulate:
30
```

- After that a graph and simulation will appear:



```

RTT 0: cwnd = 1, ssthresh = 32
Slow Start: cwnd doubled to 2
RTT 1: cwnd = 2, ssthresh = 32
Slow Start: cwnd doubled to 4
RTT 2: cwnd = 4, ssthresh = 32
Slow Start: cwnd doubled to 8
RTT 3: cwnd = 8, ssthresh = 32
Slow Start: cwnd doubled to 16
RTT 4: cwnd = 16, ssthresh = 32
Slow Start: cwnd doubled to 32
RTT 5: cwnd = 32, ssthresh = 32
Congestion Avoidance: cwnd increased to 33
RTT 6: cwnd = 33, ssthresh = 32
Congestion Avoidance: cwnd increased to 34
RTT 7: cwnd = 34, ssthresh = 32
Triple Duplicate ACK at RTT 7: ssthresh set to 17, cwnd set to 17
RTT 8: cwnd = 17, ssthresh = 17
Congestion Avoidance (post-loss): cwnd increased to 18
RTT 9: cwnd = 18, ssthresh = 17
Congestion Avoidance: cwnd increased to 19
RTT 10: cwnd = 19, ssthresh = 17
Congestion Avoidance: cwnd increased to 20
RTT 11: cwnd = 20, ssthresh = 17
Congestion Avoidance: cwnd increased to 21
RTT 12: cwnd = 21, ssthresh = 17
Timeout at RTT 12: ssthresh set to 10, cwnd set to 1
RTT 13: cwnd = 1, ssthresh = 10

```

4. Code Structure:

```

if random.random() < p:
    if random.random() < 0.5:
        ssthresh = max(cwnd // 2, 2)
        cwnd = ssthresh
        loss_rtts.append(rtt)
        loss_types.append(0)
        in_recovery = True
        print(f"{RED}Triple Duplicate ACK at RTT {rtt}: ssthresh set to {ssthresh}, cwnd set to {cwnd}")
    else:
        ssthresh = max(cwnd // 2, 2)
        cwnd = 1
        loss_rtts.append(rtt)
        loss_types.append(1)
        in_recovery = True
        print(f"{RED}Timeout at RTT {rtt}: ssthresh set to {ssthresh}, cwnd set to {cwnd}{RESET}")

```

```

else:
    if cwnd < ssthresh:
        next_cwnd = cwnd * 2
        if next_cwnd < ssthresh:
            cwnd = next_cwnd
            print(f"{YELLOW}Slow Start: cwnd doubled to {cwnd}{RESET}")
        else:
            if not is_power_of_2(ssthresh):
                largest_power = 1 << int(math.log2(ssthresh))
                if cwnd == largest_power:
                    cwnd = ssthresh
                    print(f"{YELLOW}Slow Start: cwnd set to ssthresh = {cwnd} (transition to )
                elif cwnd < largest_power:
                    cwnd = next_cwnd
                    print(f"{YELLOW}Slow Start: cwnd doubled to {cwnd}{RESET}")
            else:
                cwnd = next_cwnd
                print(f"{YELLOW}Slow Start: cwnd doubled to {cwnd}{RESET}")

```

$$\text{cwnd}(t+1) = \begin{cases} 2 \times \text{cwnd}(t) & \text{if } \text{cwnd}(t) < \text{ssthresh} \text{ (Slow Start)} \\ \text{cwnd}(t) + 1 & \text{if } \text{cwnd}(t) \geq \text{ssthresh} \text{ (Congestion Avoidance)} \\ \text{ssthresh} = \max(\text{cwnd}(t)/2, 2) & \text{on Triple Duplicate ACK} \\ \text{cwnd}(t) = \text{ssthresh} & \text{after Triple Duplicate ACK} \\ \text{ssthresh} = \max(\text{cwnd}(t)/2, 2) & \text{on Timeout} \\ \text{cwnd}(t) = 1 & \text{after Timeout} \end{cases} \quad (1)$$

TCP_TAHOE_SIMULATION-

1. Background:

The TCP Tahoe simulation provides valuable insights into the behavior of one of the foundational congestion control algorithms in computer networking. By visualizing the changes in congestion window over time and under various loss scenarios, the simulation helps build intuition about how TCP adapts to network conditions.

2. TCP Tahoe Algorithm Overview:

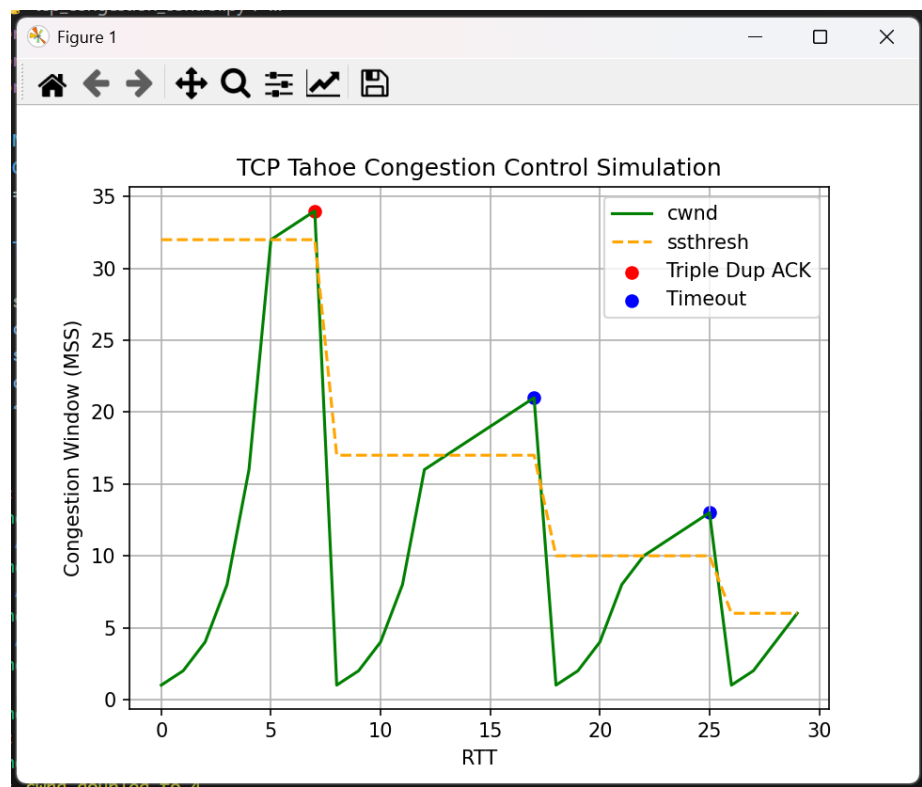
TCP Tahoe implemented three key mechanisms that revolutionized network congestion management:

- Slow Start: Exponential growth of the congestion window until reaching the slow start threshold (ssthresh)
- Congestion Avoidance: Linear growth of the congestion window after reaching ssthresh
- Fast Retransmit: Detecting packet loss through triple duplicate ACKs and responding with congestion window reduction

A critical characteristic of TCP Tahoe is that it responds to all loss events by resetting the congestion window to 1 MSS and entering slow start, regardless of whether the loss was detected via timeout or triple duplicate ACKs

3. How to operate:

- The way to operate this is the same as Reno-



```

RTT 0: cwnd = 1, sssthresh = 32
Slow Start: cwnd doubled to 2
RTT 1: cwnd = 2, sssthresh = 32
Slow Start: cwnd doubled to 4
RTT 2: cwnd = 4, sssthresh = 32
Slow Start: cwnd doubled to 8
RTT 3: cwnd = 8, sssthresh = 32
Slow Start: cwnd doubled to 16
RTT 4: cwnd = 16, sssthresh = 32
Slow Start: cwnd doubled to 32
RTT 5: cwnd = 32, sssthresh = 32
Congestion Avoidance: cwnd increased to 33
RTT 6: cwnd = 33, sssthresh = 32
Congestion Avoidance: cwnd increased to 34
RTT 7: cwnd = 34, sssthresh = 32
Triple Duplicate ACK at RTT 7: sssthresh set to 17, cwnd set to 1
RTT 8: cwnd = 1, sssthresh = 17
Slow Start: cwnd doubled to 2
RTT 9: cwnd = 2, sssthresh = 17
Slow Start: cwnd doubled to 4
RTT 10: cwnd = 4, sssthresh = 17
Slow Start: cwnd doubled to 8
RTT 11: cwnd = 8, sssthresh = 17
Slow Start: cwnd doubled to 16
RTT 12: cwnd = 16, sssthresh = 17
Slow Start: cwnd set to sssthresh = 17 (transition to congestion avoidance)
RTT 13: cwnd = 17, sssthresh = 17

```

4. Code Structure:

- Exactly same as reno, only difference:

```

if random.random() < p:
    if random.random() < 0.5:
        sssthresh = max(cwnd // 2, 2)
        cwnd = 1 # Changed from cwnd = sssthresh for Tahoe
        loss_rtts.append(rtt)
        loss_types.append(0)

```